Szegedi Tudományegyetem

Informatikai Intézet

SZAKDOLGOZAT

Szabó Ádám

2026

Szegedi Tudományegyetem
Informatikai Intézet

TuneTrivia:
React és Java Spring alapú zenei kvíz

Szakdolgozat

Készítette:
Szabó Ádám
programtervező
informatikus szakos
hallgató

Témavezető:
Dr. Bilicki Vilmos
egyetemi adjunktus

Szeged
2026

Feladatkiírás

Az interneten nehezen található olyan webalkalmazás, ami elsősorban a zenerajongoknak és a diákoknak segít interaktív módon fejleszteni zenei tudásukat. A feladat egy olyan webalkalmazás készítése Vite + React frontend és Java Spring Boot backend használatával, ahol a felhasználók zenei kvízeken keresztül próbálhatják meg felismerni a dalokat és előadókat. A játékosok különböző műfajok közül választhatnak (Pop, Rock, Hip-Hop, Jazz, Klasszikus, Elektronikus, Reggae, Latin), hogy személyre szabott játékélményt kapjanak. A felhasználók képesek klasszikus és végtelen játékmódok között választani, ahol a klasszikus mód egy előre meghatározott számú dalból áll, a végtelen mód pedig addig tart, amíg egy hibás választ nem adnak. Az alkalmazás Last.fm integrációval rendelkezik, amely lehetővé teszi a felhasználók számára saját hallgatási előzményeik importálását. A játékhoz tartozik Deezer API integráció, amely friss és aktuális dalokat biztosít. A pontszám és a sorozat (streak) nyomon követése, valamint a helyes és helytelen válaszokhoz tartozó hangjelzések motiválják a játékosokat. A felhasználók a Google OAuth segítségével regisztrálhatnak, és az alkalmazás rögzíti az összes játékadatukat, lehetővé téve a teljesítmény és a fejlődés hosszú távú nyomon követését.

Tartalmi összefoglaló

A téma megnevezése:

TuneTrivia - Zenei Kvíz Webalkalmazás

A megadott feladat megfogalmazása:

A feladat egy olyan webalkalmazás létrehozása, amelyen a felhasználók gyorsan és egyszerűen tudnak zenei kvízeken keresztül fejleszteni zenei tudásukat, különböző műfajokból választani, saját hallgatási előzményeiket importálni, és közösen tanulni a zenei közösséggel. A játékosok pontszámaikat, sorozataikat és teljesítményüket nyomon követhetik, hosszú távú fejlődésüket dokumentálhatják.

A megoldási mód:

A feladatot megoldásához JavaScript/React és Java nyelveket fogom használni, valamint a Deezer API és Last.fm API szolgáltatásait az aktuális zenei adatok lekérésére. Az alkalmazás felhő alapú infrastruktúrát használ az adatok tárolásához és feldolgozásához.

Alkalmazott eszközök, módszerek:

A fejlesztés Visual Studio Code és IntelliJ IDEA fejlesztőkörnyezetekben történik. A fejlesztés során React keretrendszert (frontend) és Spring Boot keretrendszert (backend) fogom használni JavaScript/TypeScript és Java nyelvekkel, valamint a PostgreSQL adatbázissal és Docker containerizációval. Az alkalmazás Google OAuth 2.0 hitelesítést alkalmaz a biztonságos felhasználói azonosításhoz.

Elért eredmények:

A webalkalmazást Docker Compose segítségével containerizáltam, amely lehetővé teszi az egyszerű telepítést és futtatást különböző környezetekben. Az alkalmazás a Deezer API és Last.fm API integrációval rendelkezik, személyre szabott és dinamikus tartalmakat biztosít.

Kulcsszavak:

zenei kvíz, tanulás, interaktív, React, Spring Boot, Deezer API, Last.fm API, PostgreSQL, Docker, Google OAuth, teljesítményelemzés, reszponzív design

Tartalomjegyzék

Tartalom

Tartalmi összefoglaló 2

Tartalomjegyzék 3

Bevezetés 6

1. PIACKUTATÁS 7

1.1. Bandle 7

1.2. Songless 8

1.3. Crosstune 8

2. FUNKCIONÁLIS SPECIFIKÁCIÓ 9

2.1. Felhasználók két csoportja 9

2.2. Vendég felhasználó 9

2.3. Vendég felhasználó által elérhető funkciók 9

2.4. Regisztrált felhasználó 10

2.5. Regisztrált felhasználó által elérhető funkciók 10

2.6. Játékmódok: 10

2.7. Statisztika és Adatrögzítés 10

2.8. Részletes statisztikák 10

2.9. Last.fm Integráció 10

2.10. Profil Kezelés 11

2.11.1 TÁBLÁZAT: RENDSZER FUNKCIÓK 11

3. MEGJELENÉS 11

3.1. Főoldal 12

3.2. Regisztráció, bejelentkezés 12

3.3. Kvízjáték közbeni oldal 13

3.4. A játék végének összefoglalója 14

3.5. Profilmenü 15

4. FELHASZNÁLT TECHNOLÓGIÁK 16
4.1. React 16
4.2. Vite 16
4.3. Spring Boot 16
4.4. Spring Security és JWT 16
4.5. PostgreSQL 17
4.6. Flyway 17
4.7. Deezer API 17
4.8. Last.fm API 17
4.9. React H5 Audio Player 17
4.10. Google bejelentkezés 18
5. ARCHITEKTÚRA 18
5.2. Backend 18
5.3. Kommunikáció a frontend és backend között 19
6. A RENDSZER KIALAKÍTÁSA 19
6.1. Komponensek 19
6.2. SZOLGÁLTATÁSOK 21
6.3. Adatszerkezetek és modell osztályok 22
6.4. Routing 23
7. BIZTONSÁG 23
8. Adatmodell 25
8.1. Játékstatisztika (PlayStat) 25
8.3. Felhasználói adatok (UserInfo) 25
9. A RENDSZER MAGAS SZINTŰ FOLYAMATAI 26
10. FONTOSABB KÓDRÉSZLETEK 26
10.1. Bejelentkezés 26
10.2. Kvíz indítása és játékmenet 27
10.3. Eredmények mentése és statisztikák kezelése 28

11.TAPASZTALATOK, TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK 30

12. Mesterséges intelligencia használata a fejlesztés során 31

12.1. Frontend kódgenerálás és validálás 31

12.2. Backend és Java Spring kódgenerálás 32

12.3. Dokumentáció és szöveges tartalom 32

12.4. Problémák az AI generált kódrészletekben, szövegekben 32

IRODALOMJEGYZÉK 33

KÖSZÖNETNYILVÁNÍTÁS 35

ELEKTRONIKUS MELLÉKLETEK 36

Bevezetés

A modern oktatás és a zeneoktatás fejlesztése számos kihívással szembesül. A hallgatók és diákok sokszor passzívan tanulnak a tananyagokból, anélkül, hogy valós motiváció lenne az ismeretek elsajátítása mögött. A hagyományos tanítási módszerek mellett a hallgatók számára szükséges egy olyan aktív, szórakoztató megközelítés, amely valódi elkötelezettséget és érdeklődést ébreszt a tanulás iránt. A zenei ismeretek fejlesztése különösen fontos a különböző oktatási szinteken, azonban a rendelkezésre álló eszközök gyakran túlméretezett vagy nem elég vonzó a fiatalabb generáció számára.

A TuneTrivia webalkalmazás ennek a problémának a megoldására készült, ahol a felhasználók interaktív zenei kvízeken keresztül fejleszthetik szórakoztatás közben zenei tudásukat. Az alkalmazás lehetővé teszi a diákok és hallgatók számára, hogy különböző műfajokból válasszanak, személyre szabott tanulási élményt kapjanak, és folyamatosan fejlesszék zenei hallásuk és ismereteiket. Fontos szempont volt az alkalmazás könnyű és gyors használata, hogy a felhasználók ténylegesen a zenei ismeretek fejlesztésére tudjanak koncentrálni.

Az alkalmazás több játékmódot támogat: a klasszikus módban a felhasználó egy előre definiált számú dalból álló kvízkörön megy át, a végtelen módban pedig addig játszik, amíg egy hibás választ nem ad. A felhasználók saját Last.fm profiljaikból importálhatják kedvelt dalaikat, ezáltal személyes, motiváló tanulási élményt kapnak. A valós idejű pontszámkövetés és hangjelzések azonnali visszajelzést adnak, motiválva a játékosokat jobb teljesítményre.

Az alkalmazás a felhasználók teljes játékéletét rögzíti: mely dalokat ismerte fel, melyekben hibázott, mely műfajok az erősségei. Ez az adatrögzítés lehetővé teszi a hallgatók számára, hogy hosszú távú fejlődésüket elemezzék, azonosítsák tanulási gyengeségeiket, és célzott gyakorlásra koncentrálhassanak. A Google OAuth integrációnak köszönhetően a felhasználók könnyedén hozzáférhetnek saját profiljukhoz és statisztikáikhoz.

A TuneTrivia szórakoztató, de mégis tudást fejlesztő platformként funkcionál, amely különösen vonzó a zenerajongók, diákok és hallgatók számára. A modern, reszponzív felhasználói felület biztosítja, hogy az alkalmazás asztali számítógépen, laptopon, tableten és mobilon egyaránt zökkenőmentesen működik, ezáltal egy szélesebb csoportnak nyújt hozzáférést az interaktív zenei tanuláshoz. Manapság a közösségi média és a szocializációs aspektusok fontos motivációs tényezők, és a TuneTrivia statisztikavezérlési rendszere lehetővé teszi a felhasználók számára, hogy lekövessék fejlődésüket és büszke legyenek elért eredményeikre.

1. Piackutatás

A TuneTrivia fejlesztésének legelején területi áttekintést, piackutatást végeztem inspirálódásként. Az áttekintés során olyan webalkalmazásokat kerestem és tekintettem meg, amelyeknek célja az interaktív zenei kvízélmény nyújtása a felhasználók számára. Az alábbi táblázat a területi áttekintés során készült összefoglaló táblázat, amely az áttekintett webalkalmazások funkcionalitását foglalja össze. A TRUE jelzi, ha a webalkalmazásban megtalálható az adott funkcionalitás, a FALSE jelzi, ha nem található meg az adott funkcionalitás.

Funkcionalitás	Bandle	Songless	Crosstune	TuneTrivia
Interaktív zenei kvíz	TRUE	TRUE	TRUE	TRUE
Napi kihívás (Daily)	TRUE	TRUE	TRUE	FALSE
Sztemek/Rétegek felépítése	TRUE	FALSE	FALSE	FALSE
Vokálok nélküli változat	FALSE	TRUE	FALSE	FALSE
Crossword/Puzzle mechanika	FALSE	FALSE	TRUE	FALSE
Pontozás és statisztika	TRUE	TRUE	TRUE	TRUE
Felhasználói profilok	TRUE	TRUE	TRUE	TRUE
Többféle játékmód	TRUE	TRUE	FALSE	TRUE
Műfaj szerinti szűrés	TRUE	TRUE	FALSE	TRUE
Last.fm integráció	FALSE	FALSE	FALSE	TRUE
Deezer API integráció	FALSE	FALSE	FALSE	TRUE
Hosszú távú statisztika-követés	TRUE	TRUE	TRUE	TRUE
Reszponzív design	TRUE	TRUE	TRUE	TRUE
Google OAuth hitelesítés	TRUE	TRUE	TRUE	TRUE
Multiplayer/Verseny mód	TRUE	FALSE	FALSE	FALSE

1.1. Bandle

A Bandle (bandle.app) egy népszerű zenei puzzle játék, amely a Wordle formátumára épít, de zenei azonosítási kihívásokkal. A játék egyedi megközelítése az, hogy a zenét sztemek (egyes hangszerek) révén építi fel: először csak a dobok hallhatók, majd szekvenciálisan hozzáadódnak a basszus, gitár, billentyűzet és a teljes hangszerelés. A játékos feladata, hogy minél kevesebb réteg hallgatása után felismerje az adott dalt. A Bandle napi kihívásokat kínál, amelyekben az egész világon ugyanaz a dal van minden nap, valamint különféle "song pack"-eket kínál a múlt évtizedei és műfajai szerint kategorizálva. A multiplayer mód lehetővé teszi a barátokkal való

versengést. A játék azonban korlátozottabb a műfaj szerinti szűrés tekintetében, és nem támogatja a felhasználók személyes hallgatási előzményeinek importálását.

1.2. Songless

A Songless (songless.app) egy másik zenei azonosítási játék, amely speciális módon különbözik a Bandle-től: teljes egészében vokálok nélkül működik. Míg a Bandle rétegről rétegre építi fel a zenét, a Songless már az első hallgatástól a dalok tisztán instrumentális változatát kínálja. A játék célja, hogy a felhasználók egy limitált számú próbálkozáson belül felismerjék a dalt vagy az előadót. Minden helytelen próbálkozás egy kicsit hosszabb vagy részletesebb verzióját jeleníti meg az instrumentális tracknek. A Songless különösen népszerű a zenei producerek, a trivia nerdk és a fókuszált tanulásra törekvő felhasználók körében, mivel az agyedzésre és a magasabb nehézségi szintre fókuszál. A játéknak azonban korlátozottabb műfaj-szűrési lehetőségei vannak, és nem nyújt Last.fm integrációt vagy személyre szabott tartalom importálási lehetőségeket.

1.3. Crosstune

A Crosstune (crosstune.io) egy innovatív zenei crossword játék, amely a hagyományos keresztrejtvény logikáját kombinálja az audio-alapú zenei triviával. A játékban a felhasználók egy rácsot tölthetnek ki, ahol az egyes szavakhoz audio klipek kapcsolódnak. A játékos feladata, hogy felismerje az audio klipből a dalt vagy az előadót, és a helyes választ beírja a rácsba. Ahogy egy igazi keresztrejtvénynél, az egyes válaszok betűi más szavakkal metszik egymást, amely segít a felhasználónak szűkíteni a lehetőségeket. A Crosstune napi puzzle-kat kínál, valamint tematikus boardokat, amelyek speciális előadókra vagy műfajokra fókuszálnak. Ez a kombinált megközelítés egy magasabb szintű kihívást nyújt, mivel a felhasználónak nem csupán egy dalt kell felismernie, hanem 10-15 dalt is, és azokat összekapcsolt puzzle-ként kell megoldania. A Crosstune azonban nem támogatja a műfaj szerinti szűrést, a Last.fm integrációt vagy a végtelen játékmódot.

2. Funkcionális specifikáció

A TuneTrivia webalkalmazás funkcionalitásait és a felhasználók közötti kapcsolatokat szöveges formában mutatjuk be. Az aktorok a felhasználók, akik valamilyen módon interakcióba lépnek a rendszerrel, míg a funkciók maguk a rendszer által kínált lehetőségek.

2.1. Felhasználók két csoportja

A webalkalmazás felhasználóit két csoportba sorolhatjuk: vendég felhasználók és regisztrált fiókkal rendelkező felhasználók. A két csoport közötti különbség jelentős, hiszen a regisztrált fiókkal rendelkező felhasználók sokkal több funkciót érnek el, mint az oldalra bejelentkezés nélkül játszani próbáló felhasználók.

2.2. Vendég felhasználó

A vendég felhasználóknak a webalkalmazásban lehetőségük van regisztrálni Google OAuth segítségével, illetve amennyiben van már regisztrált fiókjuk, lehetőségük van bejelentkezni. Ha továbbra is vendég felhasználóként szeretnék használni a webalkalmazást, akkor a lehetőségek száma korlátozott.

2.3. Vendég felhasználó által elérhető funkciók

- Műfaj kiválasztása (Pop, Rock, Hip-Hop, Jazz, Klasszikus, Elektronikus, Reggae, Latin)
- Klasszikus játékmód játszása (előre meghatározott számú dal)
- Végtelen mód játszása (addig játszik, amíg hibásan nem válaszol)
- Zenei preview hallgatása az alkalmazáson belül
- Hangerő szabályozása
- Skip/Jump funkció használata (a zenei preview különböző pontjaira való ugrás)
- Valós idejű pontszám megtekintése a játék során

Azonban a vendég felhasználók nem képesek:

- Az eredményeik mentésére
- Statisztikáik nyomon követésére

2.4. Regisztrált felhasználó

A vendég felhasználókhoz képest a bejelentkezett felhasználók korlátozás nélkül hozzáférnek a webalkalmazás összes funkcionalitásához.

2.5. Regisztrált felhasználó által elérhető funkciók

2.6. Játékmódok:

Az összes vendég felhasználó által elérhető funkció (műfaj kiválasztása, játékmódok, hangerő-szabályozás, skip funkció)
Klasszikus és végtelen játékmódok teljes hozzáférése

2.7. Statisztika és Adatrögzítés

Az alkalmazás automatikusan rögzíti az összes játékadatot, beleértve:
Egyes dalok helyes/helytelen válaszait
Az előadók felismerésének sikerességét
Az elért pontszámokat
A skip-ek számát
A felhasználók megtekinthetik az összes eddigi játékadatukat a profil menüben

2.8. Részletes statisztikák

Az adott dalok és előadók teljesítményelemzését
A skip-ek számának nyomon követését

2.9. Last.fm Integráció

Saját Last.fm top trackjeinek importálása
Last.fm playlistek importálása

2.10. Profil Kezelés

Kijelentkezés a Google fiókból

Funkció	Vendég	Regisztrált
Műfaj kiválasztása	✓	✓
Klasszikus játékmód	✓	✓
Végtelen játékmód	✓	✓
Hangerő szabályozása	✓	✓
Zenei preview hallgatása	✓	✓
Skip / Jump funkció	✓	✓
Pontszám megtekintése (live)	✓	✓
Statisztikák megtekintése és mentése	✗	✓
Last.fm integráció	✓	✓
Teljesítmény elemzése	✗	✓
Google bejelentkezés	✓	✓
Kijelentkezés	✗	✓

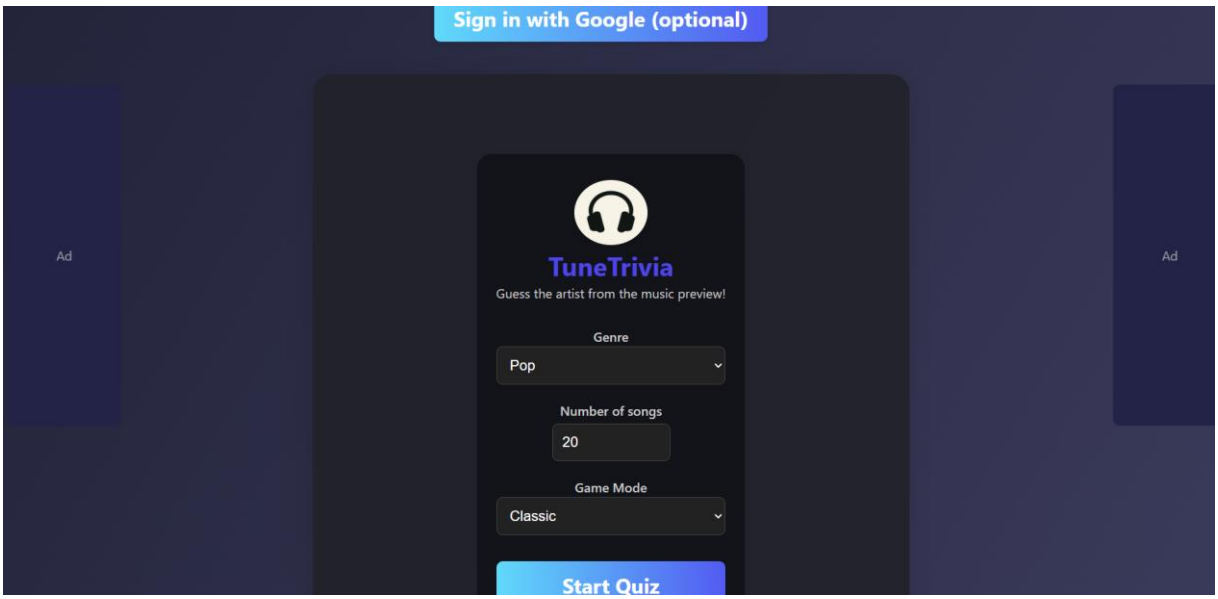
2.11.1 táblázat: Rendszer funkciók

3. Megjelenés

A webalkalmazás felhasználói felületének tervezése során elsődleges szempont volt az átlátható, gyorsan használható megjelenés kialakítása. A kliensoldali felület React és Vite környezetben készült, a backend pedig Spring Boot alapokon biztosítja az adatkezelést és az üzleti logikát. A felhasználói élmény javítása érdekében egy minden fényviszonyban kellemes hátteret választottam. A tervezés és fejlesztés során kiemelt figyelmet kapott a reszponzív működés, ezért a felületet több mobil- és tabletméretre is optimalizáltam. A különböző képernyőméretekhez igazodó elrendezéseket CSS media query-k és breakpoint-alapú stílusok segítségével valósítottam meg, hogy a tanulási élmény minden eszközön egységes és jól kezelhető maradjon.

3.1. Főoldal

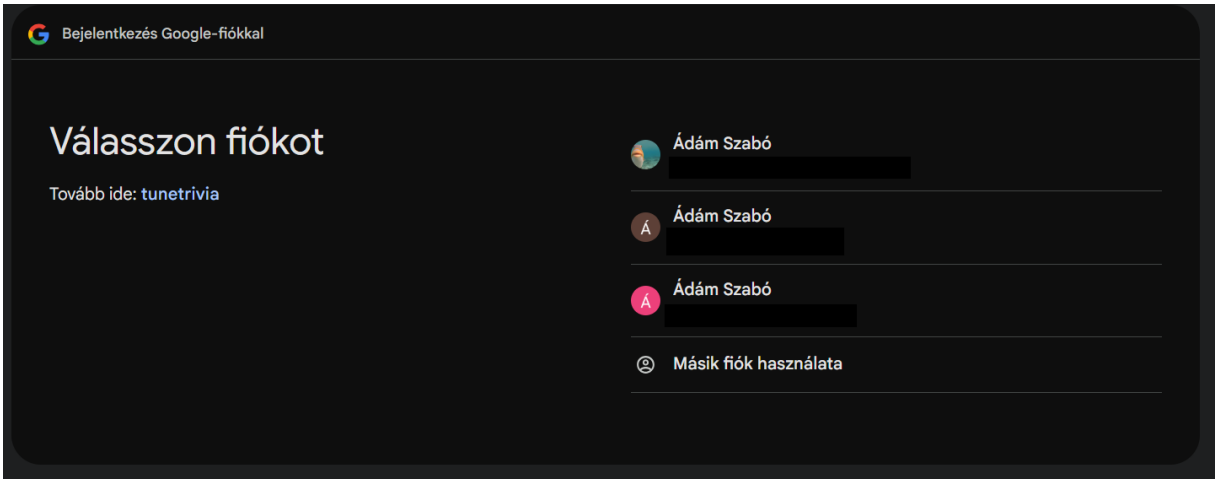
A felhasználók a weboldalra látogatáskor először a főoldalra jutnak, itt lehet Google fiókkal bejelentkezni, illetve elindítani játékot. A játékmódokat legördülő menüben lehet kiválasztani. Be tudnak jelentkezni a Google fiókjukba, a fenti gombbal. Jobbra és balra reklámfelület jelenik meg, telefonon ez csak alul jelenik meg.



3.1.1. ábra: A fő oldal megjelenése

3.2. Regisztráció, bejelentkezés

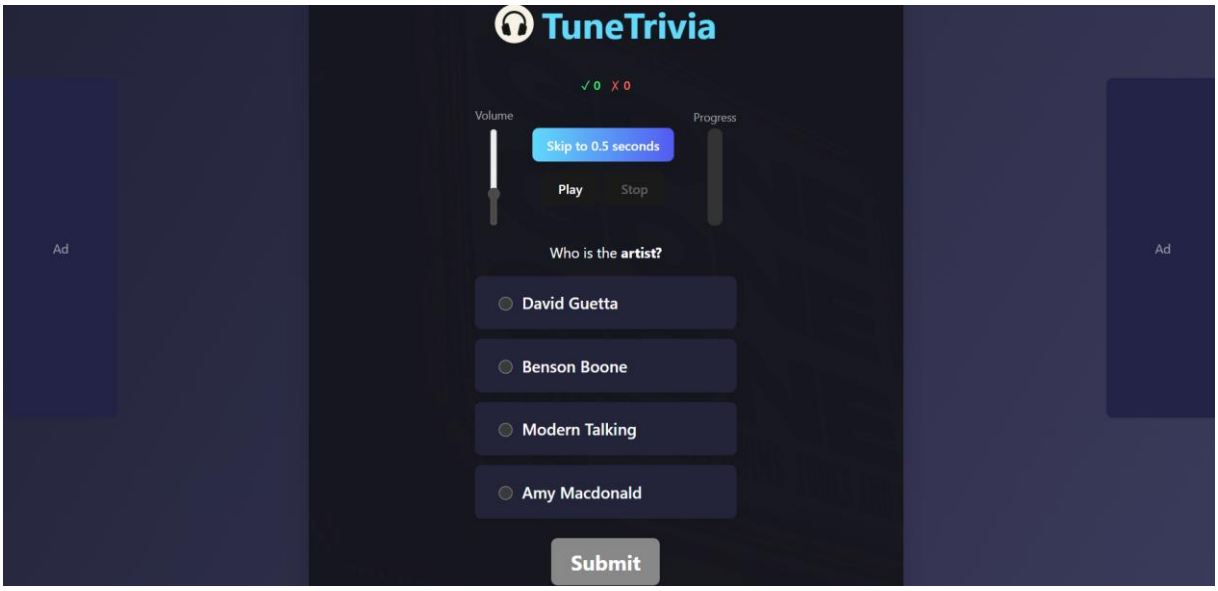
A regisztráció és bejelentkezés Google OAuth rendszerrel van lefejlesztve, így biztosítva az egyszerű ki és be jelentkezést.



3.2.1. ábra: Google regisztráció

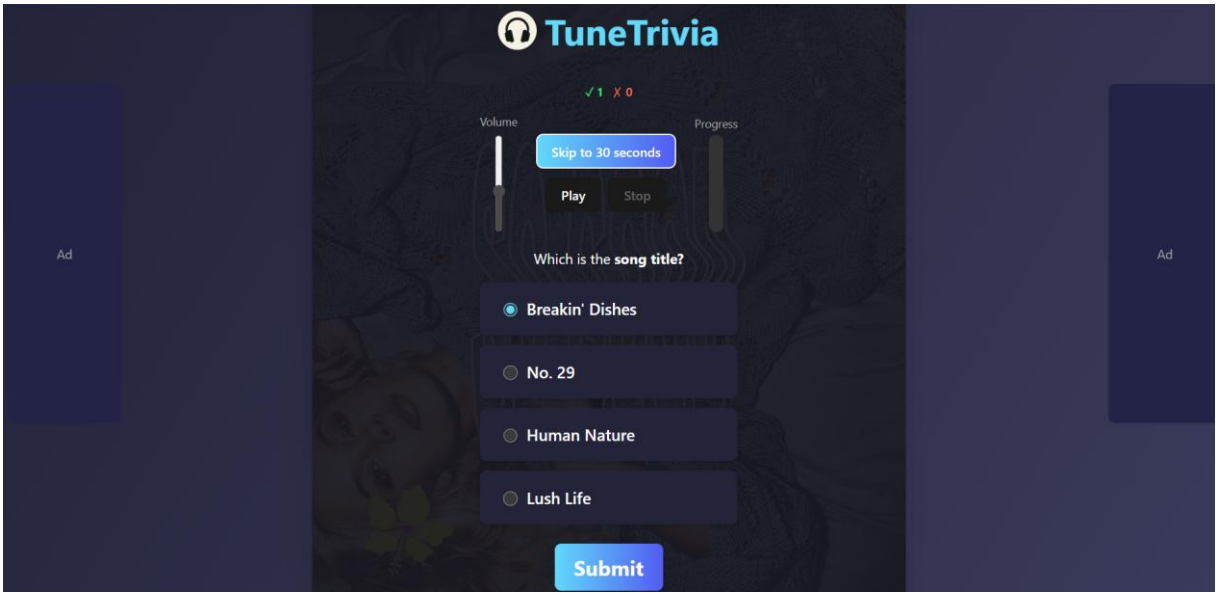
3.3. Kvízjáték közbeni oldal

A kvízjáték közben sok opció nyílik meg a felhasználó felé. Felhangosíthatja, és lehalkíthatja a zenét, továbbmehet a következő snitt lejátszására a zenéből, kiválaszthatja a helyes megoldást, megállíthatja a zenét, ha úgy tartja kedve. Jobb oldalt megjelenik egy függőleges státuszcsík, ami a zenéből eltelt időt jelzi, 30 másodpercnél lesz teljesen betöltve. A háttérben megjelenik az adott zeneszám album borítója, viszont eleinte nagyon sötét árnyalatban, nehezen felismerhetően. Minél több snittet játszik le a felhasználó, annál világosabb lesz, ezzel segítve a zene felismerését. Felül zöld pipa és piros kereszt jelzi a jelenlegi állást.



3.3.1. ábra: Kvíz közbeni oldal felülete

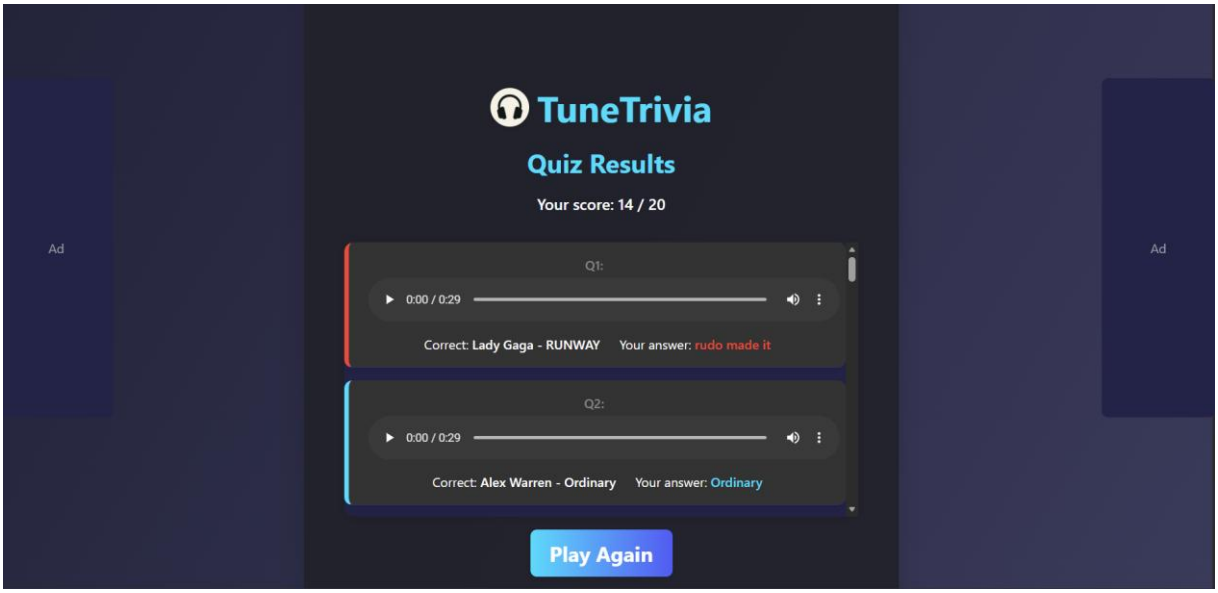
Ezek mellett minél tovább megyünk a kérdésekbe, annál jobban betölt a felső csík, ami minden egyes válasz után töltődik, és a kvíz végére teljesen betölt. Minden helyes és helytelen válasz után hangeffekt játszódik le, ezeket a frontend public mappájában tárolom correct.waw és wrong.waw néven. Ezek a snittek állnak a felhasználó rendelkezésére: 0,1 másodperc, ez az alapértelmezett, innen kezdődik a játék, ezután 0,5 másodperc, 1 másodperc, 5 másodperc, 10 másodperc, és végül 30 másodperc, ami egyébként a Deezertől kapott hangfájlok limite.



3.3.2. ábra: Az albumborító világosabb, illetve már 30 másodperces a snitt

3.4. A játék végének összefoglalója

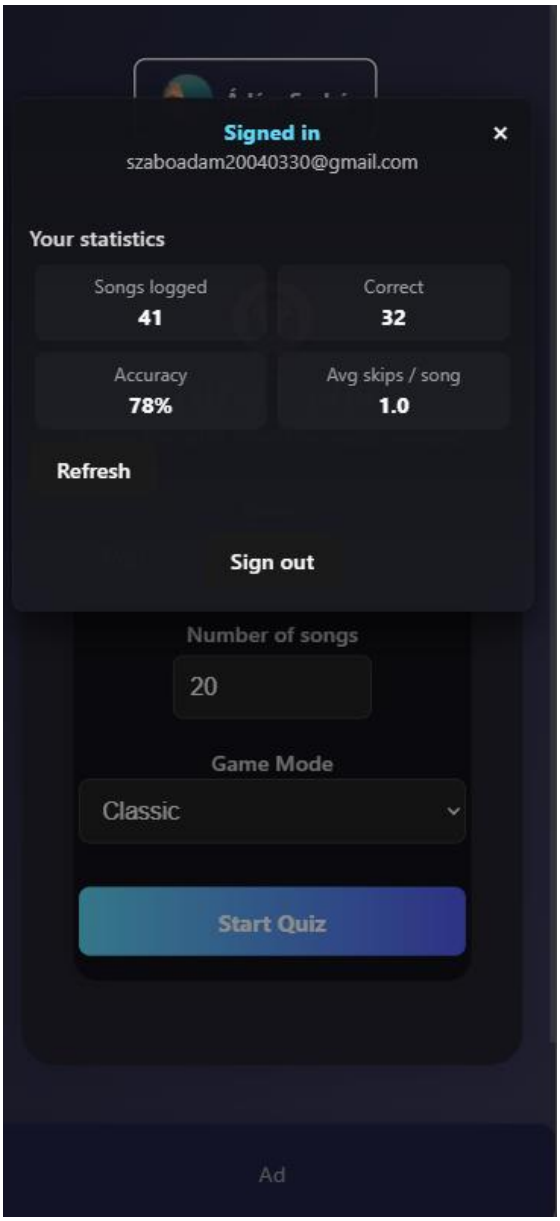
A játék végén megjelenik, hogy a rendelkezésre álló zeneszámokból mennyit talált el a felhasználó, illetve vissza tudja hallgatni ezeket a zeneszámokat. Egy görgethető felület áll rendelkezésre a zenék visszahallgatásához, és a helyes válaszok visszanézéséhez. A play again gombra kattintva a játék visszatér a főoldalra, ahol újabb játékot tud a felhasználó kezdeményezni.



3.4.1. ábra: A játék vége felülete látható egy rossz és egy jó válasszal, illetve legörgetve14 jó megoldással összesen

3.5. Profilmenü

A Google bejelentkezés után a felhasználónévre kattintva elérhetővé válik a profilmenü, amit egy felugró ablakként lehet kezelni, itt tudsz kijelentkezni, illetve megnézni a statisztikáidat. A statisztikákat PostgreSQL adatbázissal, egy összesített táblába mentem, ahol minden felhasználó játékelőzményeit tárolom. Innen egy SQL select lekérdezéssel kapja meg a felhasználó a statisztikáit, amik az alábbiak: játszott zenék száma, helyes megoldások száma, pontosság, illetve annak az átlaga, hogy mennyiszer kellett a felhasználónak átugrani a következő snittre. Ezek közül a pontosság és a snitt átugrás átlaga növeli a játék kompetitív érzését.



3.5.1. ábra: A profilmenü, illetve a mobilos megjelenítés (alul a reklámfelület)

4. Felhasznált technológiák

4.1. React

A React egy nyílt forráskódú, JavaScript alapú könyvtár, amelyet felhasználói felületek készítésére használnak. A jelen webalkalmazás frontend része React segítségével valósult meg, mivel lehetőséget biztosít a komponensalapú fejlesztésre, az újrafelhasználható elemek kialakítására, valamint a dinamikusan változó felületek egyszerű kezelésére. A React különösen előnyös olyan alkalmazások esetében, ahol a felhasználói interakciók gyakoriak és a felület állapota folyamatosan változik, mint például egy zenei kvízjátékban.

4.2. Vite

A Vite egy modern frontend build eszköz, amely gyors fejlesztői környezetet és hatékony fordítási folyamatot biztosít. A projektben a Vite felelős a React alkalmazás gyors indításáért, a fejlesztési szerver működtetéséért, valamint a produkciós build elkészítéséért. Használatával a fejlesztés során gyors újratöltés és egyszerűbb konfiguráció érhető el.

4.3. Spring Boot

A Spring Boot egy Java alapú keretrendszer, amely egyszerűbbé teszi a webalkalmazások és REST szolgáltatások létrehozását. A projekt backend része Spring Boot segítségével készült, amely biztosítja az alkalmazás üzleti logikáját, az API végpontokat, valamint a külső szolgáltatásokkal való kommunikációt. A backend feladata továbbá az autentikáció kezelése, a játékhoz kapcsolódó statisztikák feldolgozása, valamint a frontend által igényelt adatok kiszolgálása.

4.4. Spring Security és JWT

A Spring Security az alkalmazás biztonsági rétegének megvalósítására szolgál. A projektben ez a technológia felelős a felhasználói hitelesítés és jogosultságkezelés kezeléséért. A JWT tokenek segítségével a rendszer állapotmentes módon tudja kezelni a bejelentkezett felhasználókat, ami megfelelő megoldást nyújt a frontend és backend közötti biztonságos kommunikációra.

4.5. PostgreSQL

A PostgreSQL nyílt forráskódú relációs adatbázis-kezelő rendszer, amely megbízható és skálázható megoldást kínál strukturált adatok tárolására. A webalkalmazásban a játékosokhoz kapcsolódó statisztikák és egyéb perzisztens adatok tárolása PostgreSQL adatbázisban történik. Az adatbázis használata lehetővé teszi a játék eredményeinek hosszú távú megőrzését és későbbi elemzését.

4.6. Flyway

A Flyway egy adatbázis-migrációs eszköz, amely a sémaváltozások verziókövetését támogatja. A projektben a Flyway gondoskodik az adatbázis-struktúra inicializálásáról és a későbbi módosítások egységes kezeléséről. Ennek segítségével az adatbázis sémája következetesen reprodukálható különböző környezetekben is.

4.7. Deezer API

A Deezer API a zenei adatok lekérésére szolgál. Az alkalmazás ezt a külső szolgáltatást használja a játékhoz szükséges zeneszámok beszerzésére, például műfaj alapján válogatott trackek lekérésére. A backend közvetítő szerepet tölt be a frontend és a Deezer szolgáltatás között, így az adatok feldolgozása és szűrése központilag történik.

4.8. Last.fm API

A Last.fm egy zenei statisztikai és ajánlórendszer, amely a felhasználók hallgatási szokásaihoz kapcsolódó adatokat biztosít. A projektben a Last.fm integrációja lehetővé teszi további zenei listák és tartalmak bevonását a játékba, ezáltal változatosabbá téve a felhasználói élményt.

4.9. React H5 Audio Player

A React H5 Audio Player egy egyszerűen használható hanglejátszó komponens, amely HTML5 alapú audiolejátszást biztosít React környezetben. Az alkalmazásban ez a komponens felelős a zeneszámok lejátszásáért, valamint a felhasználói vezérlési lehetőségek kezeléséért, például a sávon belüli ugrás funkcióért.

4.10. Google bejelentkezés

Az alkalmazás támogatja a Google-fiókkal történő hitelesítést, amely egyszerű és felhasználóbarát belépési lehetőséget nyújt. Ennek használatával a felhasználók azonosítása biztonságosan megvalósítható, és a rendszer képes a játékoshoz kapcsolódó statisztikák tárolására is.

5. Architektúra

A webalkalmazás architektúrája két fő részből áll: frontend és backend. A két komponens egymással HTTP kéréseken keresztül kommunikál. A frontend feladata a felhasználói felület megjelenítése és a felhasználói interakciók kezelése, míg a backend az üzleti logikát, az adatkezelést, valamint a külső szolgáltatásokkal való kapcsolatot valósítja meg.

5.1. Frontend

A frontend rész a felhasználó által közvetlenül használt felületet tartalmazza. Ide tartoznak a különböző oldalak, a navigációs elemek, a játékmenethez kapcsolódó komponensek, valamint a statisztikákat megjelenítő felületek. A React komponensalapú felépítése lehetővé teszi, hogy az egyes funkciók jól elkülöníthetők és könnyen karbantarthatók legyenek. A felhasználói élmény részeként a rendszer kezeli a zenelejátszást, a válaszadást, a pontszámok nyomon követését, valamint a profilmenü megjelenítését is. A frontend feladata továbbá a backendtől kapott adatok megjelenítése és a felhasználói műveletek továbbítása a szerver felé.

5.2. Backend

A backend biztosítja az alkalmazás központi logikáját. Feladata a hitelesítés kezelése, a játékhoz szükséges adatok előállítása, a statisztikák tárolása, valamint a Deezer és Last.fm szolgáltatásokkal való kommunikáció. A backend réteg fogadja a frontendből érkező kéréseket, feldolgozza azokat, majd a megfelelő választ küldi vissza a kliensoldal számára.

Az adatbázisban tárolt információk közé tartoznak a játékosok teljesítményéhez kapcsolódó statisztikák, például a lejátszott játékok száma, a helyes válaszok aránya és egyéb mérőszámok. A backend emellett a külső API-kból érkező adatok szűrését és előkészítését is elvégzi, hogy a frontend számára csak a szükséges, használható információk kerüljenek továbbításra.

5.3. Kommunikáció a frontend és backend között

A két réteg közötti kapcsolat REST alapú HTTP kommunikációval valósul meg. Amennyiben a frontend adatot szeretne lekérni, módosítani vagy tárolni, megfelelő API hívást küld a backend felé. A backend ezekre a kérésekre válaszul adatot szolgáltat, vagy elvégzi a szükséges műveletet, például játékstatisztika mentését, felhasználói adatok kezelését vagy külső zenei források lekérését.

6. A rendszer kialakítása

A webalkalmazás frontend része React alapokon valósult meg. A React komponensalapú felépítése lehetővé teszi, hogy az alkalmazás logikája és megjelenése jól elkülöníthető, újrahasznosítható egységekből épüljön fel. A felhasználói felület működését a komponensek közötti adatátadás, az állapotkezelés, valamint a backend felé küldött HTTP kérések biztosítják. Az alkalmazás központi logikája a fő komponensekben található, míg az egyes felületi elemek külön komponensekben lettek megvalósítva.

6.1. Komponensek

A webalkalmazás több önálló React komponensből áll. Ezek egy része a fő alkalmazáslogikát kezeli, másik része pedig kisebb, újrahasznosítható felhasználói felületi elemeket valósít meg. A komponensek egymástól jól elkülönítetten működnek, így az alkalmazás karbantarthatósága és bővíthetősége egyszerűbbé válik.

A kvízjáték alkalmazásban az alábbi fő komponensek találhatók:

- *App:*

  A webalkalmazás gyökérkomponense. Ez felel a fő elrendezés összeállításáért, a bejelentkezett felhasználó adatainak lekéréséért, a statisztikák kezeléséért, valamint a Quiz, Header, GoogleLogin és ProfileMenu komponensek összekapcsolásáért.

- *Quiz:*

  Az alkalmazás legfontosabb komponense, amely a zenei kvíz teljes működését tartalmazza. Ez kezeli a műfajválasztást, a kérdések betöltését, a válaszadási logikát, a pontszámítást, a játék módjait, valamint a zenelejátszást és a kihagyás funkciót is. A komponens támogatja a klasszikus és endless játékmódot, valamint a Last.fm alapú importálást is.

- *Results:*

  A játék végén megjelenő eredményeket tartalmazó komponens. Ez felel a helyes és helytelen válaszok összegzéséért, valamint az új játék indításának lehetőségéért.

- *Header:*

  A bejelentkezett felhasználóhoz kapcsolódó fejléc megjelenítését biztosítja. Innen érhető el a profilkezelés, valamint a felhasználói menü megnyitása.

- *ProfileMenu:*

  A felhasználói profilhoz tartozó menükomponens. Ebben jelennek meg a játékos statisztikái és összegzései, illetve innen lehet kijelentkezni vagy frissíteni az adatokat.

- *GoogleLogin:*

  A Google-fiókkal történő bejelentkezésért felelős komponens. Akkor jelenik meg, amikor a felhasználó még nincs hitelesítve.

- *AdSense:*

  A hirdetési felület megjelenítésére szolgáló komponens. A webalkalmazásban ennek szerepe kiegészítő jellegű, a tartalom és az elrendezés részeként jelenik meg.

6.2. Szolgáltatások

A szolgáltatások olyan különálló egységek, amelyek egy-egy jól meghatározott feladatot látnak el. A webalkalmazásban a szolgáltatási réteg elsősorban a frontend és a backend közötti kommunikációt, valamint a külső API-k integrációját támogatja. A React alapú frontendben a komponensek közvetlenül is tartalmazhatnak logikai részeket, ugyanakkor az egységes API-hívások megvalósítására külön segédfüggvény is rendelkezésre áll. A backend oldalon a szolgáltatások felelősek az autentikációért, a játékstatisztikák kezeléséért, valamint a Deezer és Last.fm rendszerekkel való kapcsolattartásért.

A webalkalmazásban az alábbi fő szolgáltatások találhatók meg:

api.js:

A frontend és a backend közötti HTTP kommunikációt egységesen kezelő segédfüggvény. Ez felel azért, hogy a kérések a megfelelő backend címre kerüljenek elküldésre, valamint a hitelesítéshez szükséges sütik is továbbításra kerüljenek.

AuthService:

Az autentikációhoz kapcsolódó műveleteket valósítja meg. Feladata a Google-fiókkal történő bejelentkezés ellenőrzése, a tokenek feldolgozása, valamint a felhasználói adatok lekérése és kezelése. Ezen kívül a backend oldali munkamenethez szükséges biztonsági tokenek létrehozásáért is felel.

StatsService:

A játék során keletkező statisztikai adatok kezeléséért felelős szolgáltatás. Ez a komponens menti a játékos válaszaihoz kapcsolódó adatokat, valamint lekérdezi a felhasználó összesített eredményeit, például a helyes válaszok számát és az átlagos kihagyások értékét.

DeezerProxyService:

A Deezer API-val való kapcsolatot valósítja meg. Ez a szolgáltatás lekéri a műfajokhoz tartozó zeneszámokat, kiszűri a nem megfelelő adatokat, és előkészíti azokat a kvízjáték számára. Emellett gondoskodik arról is, hogy a frontend számára használható formában érkezzenek vissza a zeneszámok és az azokhoz tartozó előadók.

LastFmService:

A Last.fm rendszerből származó zenei adatok feldolgozását végzi. Feladata a felhasználó top számainak, valamint a nyilvános playlistjeinek lekérése, majd ezek összekapcsolása a Deezerből származó előnézeti zenefájlokkal. Ennek segítségével a játékhoz további egyedi zenei források is bevonhatók.

PlayStatRepository:

Bár ez nem service, hanem repository réteg, fontos szerepet tölt be az adatkezelésben. Ez az interfész felel a játékstatisztikák adatbázisból történő lekéréséért, mentéséért, valamint az összesítő lekérdezések végrehajtásáért.

A szolgáltatások együtt biztosítják, hogy a webalkalmazás logikája jól elkülönüljön a felhasználói felülettől, és az egyes feladatok átlátható, karbantartható módon legyenek kezelve.

6.3. Adatszerkezetek és modell osztályok

A webalkalmazásban az adatok kezelését különböző modell osztályok és objektumok segítik. Ezek a struktúrák határozzák meg, hogy az alkalmazásban milyen adatok tárolhatók, illetve azok milyen tulajdonságokkal rendelkeznek. A modell osztályok használata lehetővé teszi, hogy a backend által kezelt információk egységes és jól értelmezhető formában jelenjenek meg.

A projektben az alábbi fő adatmodellek találhatók meg:

PlayStat:

A játék során rögzített statisztikai adatokat leíró entitás. Tartalmazza többek között a felhasználó azonosítóját, a zeneszám műfaját, az előadót, a szám címét, a helyes vagy hibás válasz tényét, a lejátszás időpontját, valamint a kihagyások számát.

StatsSummary:

Egy összesítő modell, amely a felhasználó játékhoz kapcsolódó statisztikáit foglalja össze. Ebben szerepel a játszott körök száma, a helyes válaszok száma, valamint az átlagos kihagyások értéke.

UserInfo:

A hitelesített felhasználó adatait leíró egyszerű adatátviteli objektum. Ez tartalmazza a felhasználó azonosítóját, e-mail címét, nevét, profilképét, valamint a szerepköreit. Az adatszerkezetek használata hozzájárul ahhoz, hogy a backend által szolgáltatott információk egységesen jelenjenek meg a frontend számára, és a rendszer különböző részei között egyszerűbb legyen az adatcsere.

6.4. Routing

A webalkalmazásban a felületek közötti navigáció React alapokon, komponensváltással valósul meg. Az alkalmazás központi eleme az App komponens, amely a felhasználó aktuális állapotától függően jeleníti meg a bejelentkezési felületet, a kvízkomponenst vagy a profilhoz kapcsolódó elemeket. A navigáció során az oldalak nem külön töltődnek be, hanem a komponensek dinamikusan váltják egymást, így az alkalmazás egyetlen kliensoldali felületen belül működik. A backend oldalon a routing REST API végpontokra épül. Ezek szolgálják ki az autentikációt, a statisztikák kezelését, valamint a Deezer és Last.fm integrációhoz szükséges kéréseket. A frontend és backend közötti kommunikációt az apiFetch segédfüggvény biztosítja, amely egységes módon kezeli a HTTP kéréseket.

7. Biztonság

A webalkalmazás biztonsága a backend oldalon valósul meg. A rendszer Google OAuth alapú bejelentkezést használ, majd a hitelesített felhasználó adatai egy backend oldali tokenben és cookie-ban kerülnek kezelésre. A beérkező kéréseket egy saját szűrő ellenőrzi, amely a Bearer tokent vagy az MQ_AUTH sütit vizsgálja, és ennek alapján állítja be a felhasználó hitelesítettségét. A backend állapotmentesen működik, ezért külön session nem kerül tárolásra. A CORS beállítások csak a frontend fejlesztői címét engedélyezik, így a rendszer csak a megadott forrásból fogad kéréseket. A játékstatisztikák mentése és lekérése is csak hitelesített felhasználóhoz kötötten történik, így az adatokhoz való hozzáférés szabályozott módon valósul meg.

Erőforrás	Vendég	Bejelentkezett felhasználó
Kvízhez szükséges zeneszámok (`/deezer`)	R	R
Last.fm import adatok (`/api/lastfm/*`)	R	R
Saját felhasználói adatok (`/api/auth/me`)	-	RO
Játékstatisztika mentése (`/api/stats` POST)	-	CO
Saját statisztikák listája (`/api/stats/me`)	-	RO
Saját statisztika összegzése (`/api/stats/me/summary`)	-	RO
Bejelentkezési munkamenet/token	C	D (kijelentkezés)

7.0.1 táblázat: CRUD hozzáférés vendégnek illetve bejelentkezett felhasználónak

8. Adatmodell

Az alkalmazás adatainak kezelése a backend oldalon történik, míg a frontend a src/services/api.js fájlban található apiFetch() segédfüggvényen keresztül kommunikál a szerverrel. Ennek segítségével a HTTP kérések egységesen, a bejelentkezéshez használt cookie-val együtt kerülnek elküldésre. A rendszerben a legfontosabb perzisztált adat a játék során keletkező statisztika, emellett a felhasználói adatok és az összesített eredmények is külön modellekben jelennek meg.

8.1. Játékstatisztika (PlayStat)

A játék során minden válasz egy külön statisztikai rekordként kerül eltárolásra. A PlayStat entitás tartalmazza a játékos azonosítóját, a műfajt, a zeneszám azonosítóját, az előadót, a szám címét, a válasz helyességét, a lejátszás időpontját, valamint a kihagyások számát. Ez a modell teszi lehetővé, hogy a felhasználó korábbi játékai visszakövethetők és elemezhetők legyenek. A statisztikai adatok kezeléséért a PlayStatRepository felel, amely biztosítja a rekordok lekérését, mentését, valamint az összesítő lekérdezések végrehajtását is. 8.2. Összesített statisztika (StatsSummary) A felhasználói eredmények megjelenítéséhez az alkalmazás egy összesített modellt is használ. A StatsSummary nem önálló adatbázis-entitás, hanem a részletes játékmenet-adatokból képzett összegzés. Tartalmazza az összes lejátszott játék számát, a helyes válaszok darabszámát, valamint az egy dalra jutó átlagos kihagyások értékét.

8.3. Felhasználói adatok (UserInfo)

A hitelesített felhasználó adatait a UserInfo DTO írja le. Ebben a modellben a Google-alapú bejelentkezés során kapott alapvető információk szerepelnek, például az azonosító (sub), az e-mail cím, a név, a profilkép elérési útja, valamint a szerepkörök listája. Ez a modell az /api/auth/me végpont válaszaként jelenik meg, és a frontend ennek alapján tudja kezelni a bejelentkezett felhasználót.

9. A rendszer magas szintű folyamatai

A webalkalmazás magas szintű folyamatai közé tartozik a kvízjáték indítása, a válaszok feldolgozása, valamint az elért eredmények eltárolása. A felhasználó a fő felületen kiválasztja a kívánt játékmódot és műfajt, majd az alkalmazás a backendtől lekéri a játékhoz szükséges adatokat. A frontend és a backend közötti kommunikációt az apiFetch() segédfüggvény biztosítja, amely egységesen kezeli a HTTP kéréseket. A játék során a Quiz komponens jeleníti meg az aktuális kérdéseket, kezeli a válaszadást, valamint a játék állapotát. A felhasználó minden válasza alapján a rendszer ellenőrzi, hogy a megadott megfejtés helyes volt-e, és ennek megfelelően frissíti a pontszámot, illetve a játék menetét. A lejátszott kör végén a rendszer összegzi az eredményeket, majd a keletkezett statisztikai adatokat továbbítja a backend felé. A backend a kapott adatokat a PlayStat entitásban tárolja el, majd a statisztikák lekérését és összesítését a StatsService és a StatsController végzi. Az összesített eredmények a StatsSummary modell segítségével jelennek meg a felhasználó számára. A folyamat végén az alkalmazás visszajelzést ad a játékosnak az elért eredményről, és lehetőséget biztosít egy új játék indítására.

10. Fontosabb kódrészletek

10.1. Bejelentkezés

A TuneTrivia webalkalmazásban a felhasználók hitelesítése Google-alapú bejelentkezéssel történik. A bejelentkezési folyamatot a GoogleLogin komponens kezeli, amely a felhasználó Google-fiókjának használatával biztosítja az azonosítást. A sikeres bejelentkezés után a frontend elküldi a kapott azonosító tokent a backend felé, ahol az AuthController és az AuthService végzi el az ellenőrzést. A hitelesítés során a backend egy biztonságos tokent hoz létre, amely cookie formájában kerül vissza a klienshez. Ennek segítségével a felhasználó a továbbiakban bejelentkezett állapotban használhatja az alkalmazást. A frontend az apiFetch() segédfüggvényt használja a backenddel való kommunikációhoz, így a hitelesítési cookie minden kérésnél továbbításra kerül. A bejelentkezést követően a felhasználó adatai az /api/auth/me végponton keresztül lekérhetők, és ezek alapján jelenik meg a profilmenü, valamint a személyre szabott statisztikai információk.

```
export default function GoogleLogin() {  Show usages  Szabó Ádám *
  const backendUrl = import.meta.env.VITE_BACKEND_URL || 'http://localhost:3002';

  return (
    <div style={{ textAlign: 'center', marginBottom: 16 }}>
      <button
        className="quiz-btn"
        onClick={() : void  => {
          // start server-side OAuth flow on backend (optional)
          window.location.href = `${backendUrl}/api/auth/start`;
        }}
      >
        Sign in with Google (optional)
      </button>
    </div>
  );
}
```

10.1.1. ábra GoogleLogin komponens, ami használja a backendet

10.2. Kvíz indítása és játékmenet

A webalkalmazás fő funkciója a zenei kvíz indítása és lejátszása. A játékmenetet a Quiz komponens valósítja meg, amely kezeli a műfaj kiválasztását, a kérdések betöltését, a válaszok feldolgozását, valamint a pontszámok számítását is. A felhasználó a fő felületen kiválasztja a kívánt játékmódot, majd a komponens a backendtől lekéri a szükséges adatokat.

```
const searchTracks = async () : Promise<void>  => {  Show usages  Szabó Ádám
  setQuizEnded( value: false);
  setAnswers( value: []);
  setError( value: null);
  setStreak( value: 0);
  if (gameMode === 'classic') setBestStreak( value: 0);
  try {
    const fetchCount : number  = gameMode === 'endless' ? 100 : numSongs;
    const res : Response  = await apiFetch( path: `/deezer?genreId=${genreId}&limit=${fetchCount}`);
    if (!res.ok) return;
    const json = await res.json();
    const filterKeywords : string[]  = [
      'relaxing','white noise','therapy','nature','spa','sound','meditation','sleep','ambient','asmr','rain','ocean','thunder','instrumental','background','study',
    ];
    const isBadArtist = name => {  Show usages  Szabó Ádám
      if (!name) return true;
      if (name.length > 40) return true;
      const lower : string  = name.toLowerCase();
      return filterKeywords.some(kw : string  => lower.includes(kw));
    };
```

10.2.1. ábra Deezer search kódrész, ahol a bekért zenékből a rossz találatokat szűröm (isBadArtist), hogy ne legyenek olyan zenefájlok, amik nem illenek a játékba

10.3. Eredmények mentése és statisztikák kezelése

A játék befejezése után az elért eredmények eltárolása a backend feladata. A frontend a játék során keletkezett adatokat a StatsController megfelelő végpontjára küldi, ahol a StatsService feldolgozza azokat, majd a PlayStatRepository segítségével menti az adatbázisba. A statisztikai rekordok tartalmazzák többek között a felhasználó azonosítóját, a műfajt, a zeneszám adatait, a helyes vagy hibás választ, a lejátszás idejét, valamint a kihagyások számát. A felhasználó saját statisztikái az /api/stats/me végponton keresztül kérhetők le, míg az összesített mutatókat az /api/stats/me/summary végpont biztosítja. Ezek az adatok a StatsSummary modellben jelennek meg, amely tartalmazza az összes játszott kör számát, a helyes válaszok darabszámát, valamint az átlagos kihagyásokat. A statisztikák tárolása és lekérdezése lehetővé teszi, hogy a rendszer a későbbiekben is megőrizze a felhasználók teljesítményét, és ezek alapján összesített információkat jelenítsen meg a profilnézetben.

```
package com.tunetrivia.backend.repo;

import ...

public interface PlayStatRepository extends JpaRepository<PlayStat, Long> {  2 usages  Szabó Ádám
    List<PlayStat> findByUserId(String userId);  1 usage  Szabó Ádám

    // Count helpers
    long countByUserId(String userId);  1 usage  Szabó Ádám
    long countByUserIdAndCorrectTrue(String userId);  1 usage  Szabó Ádám

    // Compute average skips per song for a user: average of per-track averages (native SQL)
    @Query(value = "SELECT AVG(sub.avg_skips) FROM (SELECT track_id, AVG(skips) AS avg_skips FROM play_stat WHERE user_id = :userId AND skips IS NOT NULL GROUP BY track_i
    Double findAverageSkipsPerSongByUser(@Param("userId") String userId);
}
```

10.3.1. ábra PlaystatRepository, ez az interface felel az összes alap CRUD műveletért, illetve a plusz műveletekért amit beleraktam. JPA-t használ, így nem kell megírni az összes alap műveletet

```
package com.tunetrivia.backend.model;

import ...

@Data  6 usages  Szabó Ádám
@AllArgsConstructor
public class StatsSummary {
    private long totalPlays;
    private long totalCorrect;
    private double averageSkipsPerSong; // average skips aggregated per trackId
}
```

10.3.2 ábra: StatsSummary model osztály

```java
package com.tunetrivia.backend.controller;

import ...

@RestController  Szabó Ádám
@RequestMapping("/api/stats")
@RequiredArgsConstructor
@Slf4j
public class StatsController {
    private final StatsService statsService;

    @PostMapping  Szabó Ádám
    public ResponseEntity<?> saveStat(@Valid @RequestBody PlayStat stat, Authentication auth) {
        String userId = auth != null ? String.valueOf(auth.getPrincipal()) : null;
        var saved = statsService.saveForUser(userId, stat);
        return ResponseEntity.ok(saved);
    }

    @GetMapping("/me")  Szabó Ádám
    public List<PlayStat> myStats(Authentication auth) {
        String userId = auth != null ? String.valueOf(auth.getPrincipal()) : null;
        return statsService.findByUser(userId);
    }

    @GetMapping("/me/summary")  Szabó Ádám
    public ResponseEntity<StatsSummary> mySummary(Authentication auth) {
        String userId = auth != null ? String.valueOf(auth.getPrincipal()) : null;
        var summary = statsService.getSummaryForUser(userId);
        return ResponseEntity.ok(summary);
    }
}
```

10.3.2 ábra StatsController osztály, ami kezeli a statisztikai endpointok továbbküldését a servicenek

11.Tapasztalatok, továbbfejlesztési lehetőségek

A TuneTrivia fejlesztése során a technológiai döntéseket a gyors iterálhatóság és a személyes tapasztalatok határozták meg: a kliensoldali felület React + Vite környezetben készült, a szerverlogika és az API-k pedig Spring Boot alapokon valósultak meg. A frontend és a backend közötti kommunikációt a src/services/api.js fájlban található apiFetch() egységesíti, a játékmeneti rekordok perzisztenciáját a PlayStat entitás és a PlayStatRepository biztosítja, az összesített mutatókat pedig a StatsSummary DTO kezeli. Verziókezelést GitHubon végeztem.

A megvalósítás során sokat tanultam a külső zenei szolgáltatások integrációjáról: a Deezer és a Last.fm felé irányuló lekérések kezelése során külön figyelmet igényelt az autentikáció, a válaszok tisztítása és a hibakezelés (rate-limit, hibás vagy hiányos metaadatok). A hitelesítést Google OAuth alapokon valósítottam meg; a kliensoldali bejelentkezés után a backend ellenőrzi az azonosító tokent, és biztonságos cookie-kezeléssel biztosítja a felhasználói állapotot. A gyakorlatban a CORS-beállítások és a cookie attribútumok finomhangolása több iterációt igényelt, de végül stabil, fejlesztői és közeléles környezetben is használható megoldás alakult ki. A játék működését a Quiz komponens valósítja meg: a komponens kezeli a kérdések betöltését, a lejátszást, a válaszok feldolgozását és a pontszámítást; minden mentett esemény egy PlayStat rekordot eredményez, amely később elemzésre alkalmas.

A fejlesztés során kevés komoly akadályt tapasztaltam; kisebb hibák és finomhangolások adódtak (például az audio előnézet pufferelése, a külső API-k megbízhatósága, illetve bizonyos aggregációs lekérdezések hatékonysága), de ezeket rövid iterációkkal javítottam. A projekt során örömmel tapasztaltam, hogy a választott technológiák lehetővé tették a gyors prototipizálást és a könnyű moduláris fejlesztést: a React komponensek újrafelhasználhatósága és a Vite fejlesztési élménye sokat segített a gyors haladásban, a Spring Boot pedig egyszerű és robosztus keretet adott az API-k és a biztonsági réteg megvalósításához.

A leadásig nem valósítottam meg néhány további ötletet időhiány miatt; ezek közül a jövőben elsősorban a tesztelési és CI/CD-t érintő fejlesztéseket tartom érdemesnek: egység- és integrációs tesztek kiterjesztése, automatikus build és deploy folyamatok kialakítása (Docker, pipeline), valamint observability megoldások bevezetése (centralizált logok, hibamonitorozás). Multiplayer módra nem tartok igényt a jelenlegi koncepcióban, ezek helyett a legfontosabb további lépések a kódbázis stabilitásának növelése, a tesztlefedettség bővítése és a felhasználói élmény finomítása (kis méretű UX-javítások mobilon, lokalizáció szükség szerint). Hosszabb távon pedig hasznos lehet további analitikai mutatók és felhasználói statisztikák bevezetése, amelyek a játékmenetek értékelését és a funkciók priorizálását segítik.

Összegzésként elmondható, hogy a projekt sikeresen megvalósította a kitűzött célokat: egy könnyen használható, tanulást támogató zenei kvíz alkalmazás jött létre stabil backenddel és átgondolt frontenddel. A jelenlegi architektúra (React + Vite frontend, Spring Boot backend, részletes PlayStat modell) jó alapot ad a további fejlesztésekhez; a következő fontos lépések a minőségbiztosítás és az automatizált kiadási folyamatok kiépítése, valamint a felhasználói élmény kis léptékű finomítása.

12. Mesterséges intelligencia használata a fejlesztés során

A szakdolgozat készítése során a mesterséges intelligencia elsősorban GitHub Copilot (GPT-5 mini modell) a kódgenerálás és a dokumentáció támogatásában vett részt. Az AI-t iteratív munkaeszközként kezeltem: minden generált kód átnézésre, tesztelésre és szükség szerint átírásra került.

12.1. Frontend kódgenerálás és validálás

A React komponensek és a felhasználói felület fejlesztésében GitHub Copilot jelentős segítséget nyújtott, mivel az AI jelentősen sokat tanult frontend fejlesztésből. Az App, Quiz, Header és ProfileMenu komponensek alapstruktúrája, a state management és hook-ok, valamint az api.js segédfüggvény az AI javaslataiból kerültek elkészítésre. A React ismétlődő mintákat követ, ezért az AI hatékony volt az ilyen kódok gyors vázlatolásában. Azonban minden egyes komponens és függvény átnézésre kerültek. A Quiz komponens komplexebb logikáját, a játékmódok kezelése, az eredmények számítása, a statisztikák mentésének koordinálása, részben átírtam az emberi követelmények alapján. Az összes komponens manuális tesztelésre kerültek: mind a fejlesztői szerveren (Vite hot reload), mind az API-val egyesítve. Ez biztosította, hogy a UI funkcionálisan is helyesen működik.

12.2. Backend és Java Spring kódgenerálás

A backend kódban szándékosan kevesebbet használtam GitHub Copilotot. Backend fejlesztőként dolgozom, és Java Spring-gel szoros kapcsolatban állok. Az AuthController és a StatsController nagy részét én írtam, ezek az üzleti logika és biztonsági lehetőségek szerves részei. A PlayStatRepository-ban a natív SQL lekérdezés (findAverageSkipsPerSongByUser) AI javaslatából indult, de a dalonkénti átlagolás logikáját magam finomítottam. Az AuthService a Google token ellenőrzés, JWT generálás, cookie-kezelés zinte komplett emberi munka volt, mivel kritikus a biztonság. Az összes backend kód manuális tesztekre került (curl, Postman, Apidog); a felmerülő hibákat ezek szűrték ki.

12.3. Dokumentáció és szöveges tartalom

A szakdolgozat szöveges fejezetei az architektúra, az adatmodell, a kódrészletek leírása, valamint a tapasztalatok és továbbfejlesztési lehetőségek GitHub Copilot és Claude segítségével kerültek kezdeti megfogalmazásra. Az AI-nak megadtam a konkrét kódrészleteket, modell-definíciókat és az architektúrális koncepciót; az AI szöveges leírásokat generált belőlük. Az eljárás iteratív volt: az AI javaslatát én szerkesztettem, pontosítottam és feltöltettem konkrét projektrészletekkel. Minden szöveg végül saját véleményezésre és korrekciókra került; a technikai hibákat kijavítottam.

12.4. Problémák az AI generált kódrészletekben, szövegekben

Mivel az AI technológia még csak pár éve kezdte el a pályafutását, ezért még van rengeteg olyan dolog is, amiben még hiányt látok. Van egy dolog, ami kifejezetten zavar, ami pedig az, hogyha betanul a beszélgetésben egy olyan információt, ami nem igaz, akkor akármennyiszer írod le neki, hogy ez nem így van, akkor is ragaszkodik a saját álláspontjához, és ugyanazt a hülyeséget írja le újból, illetve frontend fejlesztés közben tapasztaltam, hogy a css stylingban sokszor 1 soros átírást inkább megcsinál 10 sorból, vagy kitöröl valamit, ami már működőképes volt, és semmi köze nincs a problémához. A GitHub Copilotban még az is lehetséges, hogy előző beszélgetésből szed ki információkat, amit megtanult, és ezért sokszor ki kellett törölnöm az előző beszélgetést emiatt. És emiatt sokszor nem gyorsítja, hanem inkább lassítja a munkát. Egy egyszerű hibát is képes túlbonyolítani, vagy belenyúl olyan részekbe, amikhez nem kellene. Az is gond, hogy ha egyszer rossz irányba indul el, nehezen korrigál, még akkor is, ha

jelzed neki. Emellett az sem ideális, hogy sokszor túl magabiztosan ad hibás választ, ami első ránézésre helyesnek tűnik, de valójában nem az. Emiatt különösen fontos mindent leellenőrizni, ami megint csak időt visz el.

Összességében hasznos eszköz, de még túl sok felügyeletet igényel ahhoz, hogy igazán megbízható legyen komplexebb feladatoknál.

Irodalomjegyzék

1. React: https://react.dev/ (Utolsó megtekintés: 2026. 05. 01.)
2. Vite: https://vite.dev/ (Utolsó megtekintés: 2026. 05. 01.)
3. Spring Boot: https://spring.io/projects/spring-boot (Utolsó megtekintés: 2026. 05. 01.)
4. Spring Security: https://spring.io/projects/spring-security (Utolsó megtekintés: 2026. 05. 01.)
5. Spring Data JPA:https://spring.io/projects/spring-data-jpa (Utolsó megtekintés: 2026. 05. 01.)
6. Jakarta Persistence (JPA): https://jakarta.ee/specifications/persistence/ (Utolsó megtekintés: 2026. 05. 01.)
7. Jakarta Validation: https://jakarta.ee/specifications/bean-validation/ (Utolsó megtekintés: 2026. 05. 01.)
8. PostgreSQL: https://www.postgresql.org/ (Utolsó megtekintés: 2026. 05. 01.)
9. Flyway: https://flywaydb.org/ (Utolsó megtekintés: 2026. 05. 01.)
10. Google OAuth / Google Identity: https://developers.google.com/identity (Utolsó megtekintés: 2026. 05. 01.)
11. Deezer API: https://developers.deezer.com/ (Utolsó megtekintés: 2026. 05. 01.)
12. Last.fm API: https://www.last.fm/api (Utolsó megtekintés: 2026. 05. 01.)
13. Tailwind CSS: https://tailwindcss.com (Utolsó megtekintés: 2026. 05. 01.)
14. React H5 Audio Player: https://www.npmjs.com/package/react-h5-audio-player (Utolsó megtekintés: 2026. 05. 01.)
15. Google Fonts: https://fonts.google.com (Utolsó megtekintés: 2026. 05. 01.)
16. Lombok: https://projectlombok.org (Utolsó megtekintés: 2026. 05. 01.)
17. GitHub: https://github.com (Utolsó megtekintés: 2026.05.01.)

Nyilatkozat

Alulírott Szabó Ádám programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szofverfejlesztési Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.
Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.
Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumában tárolja.

2026.05.23

Szabó Ádám

Köszönetnyilvánítás

Szeretnék köszönetet mondani elsősorban témavezetőmnek, Dr. Bilicki Vilmos egyetemi adjunktusnak, amiért végig segítette és támogatta a munkámat.

Hálával tartozom a családomnak és a barátaimnak is, akik nemcsak a szakdolgozatom elkészítése során, hanem az egész egyetemi éveim alatt mellettem álltak és mindenben támogattak. Köszönöm a türelmüket és megértésüket, amikor a közös időnkből kellett áldoznom a dolgozat előrehaladása érdekében, valamint a sok segítséget és tanácsot.

Elektronikus mellékletek

1. A webalkalmazás GitHub elérhetősége: https://github.com/adam12240/tunetrivia (Utolsó megtekintés: 2026. 05. 01.)